A 1D Finite Volume Solver in VBA: Implementation and Validation Against OpenFOAM

Abdelkader LAHMAR

July 2026

Contents

List of Figures

List of Tables

Nomenclature

Symbols

Symbol	Description	Unit
A	Face cross-sectional area	m^2
D	Diffusive conductance	$\mathrm{kg\ s}^{-1}$
F	Convective mass flux	$\mathrm{kg\ s}^{-1}$
L	Domain length	m
N	Number of control volumes	–
Pe	Cell Péclet number	–
r	Gradient ratio	–
u	Velocity	$\mathrm{m\ s}^{-1}$
x	Spatial coordinate	m
Δx	Control volume width	m
δx	Distance between neighboring nodes	m
Γ	Diffusion coefficient	$\mathrm{kg\ m}^{-1}\ \mathrm{s}^{-1}$
ϕ	Transported scalar	–
ψ	Van Leer limiter function	–
ρ	Density	$\mathrm{kg\ m}^{-3}$

Subscripts

Subscript	Meaning
P	Present (central) control volume
E	East node
W	West node
EE	Second east node
WW	Second west node
e	East face
w	West face
L	Left boundary
R	Right boundary
i	Node index

Coefficient Notation

Symbol	Description	Unit
a_P	Central coefficient	kg s^{-1}
a_E	East coefficient	kg s^{-1}
a_W	West coefficient	kg s^{-1}
a_{EE}	Second east coefficient	kg s^{-1}
a_{WW}	Second west coefficient	kg s^{-1}
S_P	Linearized source coefficient	kg s^{-1}
S_u	Constant source term	kg s^{-1}
r_e	Gradient ratio at the east face	–
r_w	Gradient ratio at the west face	–
ψ_e	Limiter value at the east face	–
ψ_w	Limiter value at the west face	–

Abbreviations

Abbreviation	Meaning
CDS	Central Differencing Scheme
UDS	Upwind Differencing Scheme
Hybrid	Hybrid Differencing Scheme
QUICK	Quadratic Upstream Interpolation for Convective Kinematics
SOU	Second-Order Upwind
MUSCL	Monotonic Upstream-Centered Schemes for Conservation Laws
FVM	Finite Volume Method
CFD	Computational Fluid Dynamics
VBA	Visual Basic for Applications

Chapter 1

Introduction

This report documents a 1D finite volume solver for the steady convection-diffusion equation, implemented in VBA within Microsoft Excel, and its validation against OpenFOAM. The solver supports eight discretization schemes: Central Differencing (CDS), Upwind Differencing (UDS), Hybrid, Second-Order Upwind (SOU), QUICK, Exponential, Power Law, and MUSCL with the Van Leer limiter.

1.1 Motivation

This project started as a personal effort to learn computational fluid dynamics by working directly from Patankar's *Numerical Heat Transfer and Fluid Flow* [1]. The original intention was modest: to compute the numerical results for the schemes derived in Chapter 3 and compare them against each other and against the exact solution derived in Chapter 2, while keeping a clear visual layout of the results. There was no intention of building a complete solver.

As more schemes and Péclet numbers were added, the need to keep results organized, comparable, and reproducible pushed the project further than originally planned. What began as a handful of manual calculations gradually became a structured program, with a proper iteration loop, convergence tracking, and a modular architecture.

1.2 Why VBA

The choice of Excel as the initial environment was not a deliberate software decision. For the purpose of computing a few cell values and inspecting them side by side, Excel was simply the most direct tool: results appear immediately in a grid, comparisons between schemes or Péclet numbers are visible, and no additional setup is needed to inspect intermediate values. Writing a Python script for the same task would have introduced more infrastructure than the task warranted at that stage.

As the project grew, Excel's built-in iterative solver proved too slow and too difficult to control, which led naturally to VBA as an extension of the same environment rather than switching to a different one. The solver's current architecture emerged over time as new problems and needs appeared step by step, eventually leading to the final version described in Chapter 4.

1.3 Validation Against OpenFOAM

Validating the solver against a reference code required choosing a tool that gives full control over the discretization scheme applied at every face.

ANSYS Fluent was the first candidate, as it is the commercial solver most commonly used in my environment. However, Fluent's implementation of higher-order schemes degrades to first-order accuracy on coarse meshes, and the limited number of schemes available to users made it difficult to reproduce the full set of discretizations derived in Chapter 3. These limitations made Fluent unsuitable as a validation tool for this work.

The second option was OpenFOAM, which gives the user full control over all discretization settings while providing a wide range of schemes and the ability to add custom schemes via a user-compiled library. This made OpenFOAM the most suitable tool for validation, Its implementation is described in Chapter 5.

Chapter 2

Governing Equations and Problem Formulation

2.1 Finite Volume Discretization

2.1.1 Internal Cells

Starting from the scalar transport equation:

$$\frac{\partial}{\partial x_i}(\rho u_i \phi) = \frac{\partial}{\partial x_i}(\Gamma_\phi \frac{\partial \phi}{\partial x_i}) \tag{2.1}$$

where ρ is the density, u_i is the velocity component, Γ is the diffusion coefficient and ϕ is the transported scalar. Applying the finite volume method to Equation (2.1) yields:

$$\sum_{faces} \rho u A \phi_{face} = \sum_{faces} \Gamma A \frac{\phi_{neighbor} - \phi_P}{\delta x} \tag{2.2}$$

where A denotes the face area. For the 1D case, the control volume has two faces: west (w) and east (e). Substituting these into the general face-summation form gives:

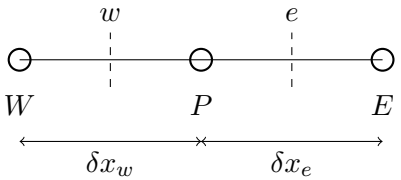


Figure 2.1: One-dimensional internal control volume.

$$\rho_e u_e A_e \phi_e - \rho_w u_w A_w \phi_w = \frac{\Gamma_e A_e}{\delta x_e}(\phi_E - \phi_P) + \frac{\Gamma_w A_w}{\delta x_w}(\phi_W - \phi_P) \tag{2.3}$$

Assuming a uniform grid with unit cross-sectional area, $\delta x_e = \delta x_w = \delta x$ and $A_e = A_w = 1$. For an incompressible isotropic fluid $\rho_e = \rho_w = \rho$ and $\Gamma_e = \Gamma_w = \Gamma$. Along with the equation of continuity:

$$\nabla \cdot u = 0 \implies u_e = u_w \tag{2.4}$$

Under these assumptions, the governing equation reduces to:

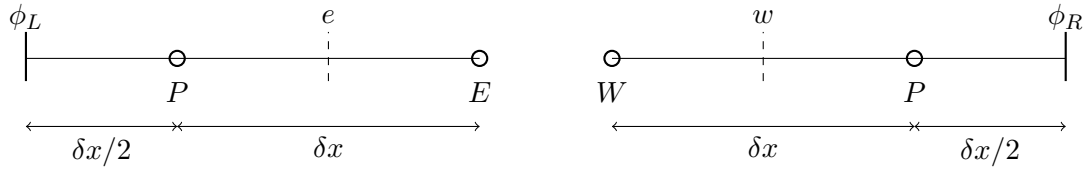
$$\rho u \phi_e - \rho u \phi_w = \frac{\Gamma}{\delta x}(\phi_E - \phi_P) + \frac{\Gamma}{\delta x}(\phi_W - \phi_P) \tag{2.5}$$

Let $F = \rho u$ and $D = \frac{\Gamma}{\delta x}$, where F denotes the convective mass flux and D the diffusive conductance.

$$F\phi_e - F\phi_w = D(\phi_E - \phi_P) + D(\phi_W - \phi_P) \tag{2.6}$$

2.1.2 Boundary Cells

The first and last control volumes require special treatment because one of their faces coincides with a domain boundary. Their geometries are illustrated in Figure 2.2. Equa-



(a) First cell (west boundary) (b) Last cell (east boundary)

Figure 2.2: One-dimensional boundary control volumes.

tion (2.2) is expanded for the first control volume as:

$$\rho_e u_e A_e \phi_e - \rho_w u_w A_w \phi_w = \frac{\Gamma_e A_e}{\delta x_e}(\phi_E - \phi_P) + \frac{2\Gamma_w A_w}{\delta x_w}(\phi_L - \phi_P) \tag{2.7}$$

Under the same assumptions:

$$\rho u \phi_e - \rho u \phi_w = \frac{\Gamma}{\delta x}(\phi_E - \phi_P) + \frac{2\Gamma}{\delta x}(\phi_L - \phi_P) \tag{2.8}$$

Defining the convective mass flux $F = \rho u$ and diffusive conductance $D = \frac{\Gamma}{\delta x}$:

$$F\phi_e - F\phi_w = D(\phi_E - \phi_P) + 2D(\phi_L - \phi_P) \tag{2.9}$$

In the same way, the equation for the last cell is:

$$F\phi_e - F\phi_w = 2D(\phi_R - \phi_P) + D(\phi_W - \phi_P) \tag{2.10}$$

2.2 Analytical Solution

2.2.1 Flux Definition

This simple 1D steady convection-diffusion problem actually has a very simple, easy-to-derive analytical solution. Starting from Equation (2.1):

$$\Gamma \frac{d^2\phi}{dx^2} - \rho u \frac{d\phi}{dx} = 0 \tag{2.11}$$

For constant Γ, ρ and u, Equation (2.11) can be written as:

$$\frac{d}{dx}(\Gamma \frac{d\phi}{dx} - \rho u \phi) = 0 \tag{2.12}$$

The flux is defined as:

$$J = \Gamma \frac{d\phi}{dx} - \rho u \phi \tag{2.13}$$

Since

$$\frac{dJ}{dx} = 0$$

the flux is constant, $J_e = J_w$.

2.2.2 First Order Differential Equation

By rearranging Equation (2.13) we get:

$$\frac{d\phi}{dx} = \frac{\rho u}{\Gamma}\phi - \frac{J}{\Gamma} \tag{2.14}$$

Writing $F = \rho u$, the equation becomes:

$$\frac{d\phi}{dx} = \frac{F}{\Gamma}\phi - \frac{J}{\Gamma}$$

This is a first order differential equation of the form $y' = ay + b$, whose general solution is $y(x) = Ce^{ax} - \frac{b}{a}$.

$$\phi(x) = Ce^{\frac{F}{\Gamma}x} + \frac{J}{F} \tag{2.15}$$

2.2.3 Boundary Conditions

Applying the following boundary conditions:

$$\phi(0) = \phi_L$$
$$\phi(L) = \phi_R$$

Then:

$$\phi_L = C + \frac{J}{F} \tag{2.16}$$

$$\phi_R = Ce^{\frac{F}{\Gamma}L} + \frac{J}{F} \tag{2.17}$$

Defining the ratio of convective to diffusive transport, known as the Péclet number P_e:

$$P_e = \frac{FL}{\Gamma}$$

Substituting this into Equation (2.17):

$$\phi_R = Ce^{P_e} + \frac{J}{F}$$

$\frac{J}{F}$ is the same in both Equation (2.16) and Equation (2.17), which allows us to solve for C:

$$\phi_L - C = \phi_R - Ce^{P_e} \tag{2.18}$$

$$\phi_L - \phi_R = C(1 - e^{P_e}) \tag{2.19}$$

$$C = \frac{\phi_L - \phi_R}{1 - e^{P_e}} \tag{2.20}$$

Substituting this into Equation (2.15):

$$\phi(x) = \frac{\phi_L - \phi_R}{1 - e^{P_e}} e^{P_e \frac{x}{L}} + \frac{J}{F} \tag{2.21}$$

Now solving Equation (2.16) for J:

$$J = F\left(\phi_L - \frac{\phi_L - \phi_R}{1 - e^{P_e}}\right) \tag{2.22}$$

Substituting this back into Equation (2.21):

$$\phi(x) = \frac{\phi_L - \phi_R}{1 - e^{P_e}} e^{P_e \frac{x}{L}} + \phi_L - \frac{\phi_L - \phi_R}{1 - e^{P_e}} \tag{2.23}$$

$$\phi(x) = \phi_L + (\phi_L - \phi_R)\frac{e^{P_e \frac{x}{L}} - 1}{1 - e^{P_e}} \tag{2.24}$$

which is identical to the expression presented by Patankar [1]:

$$\phi(x) = \phi_L + (\phi_R - \phi_L)\frac{e^{P_e \frac{x}{L}} - 1}{e^{P_e} - 1} \tag{2.25}$$

Chapter 3

Discretization Schemes

3.1 Central Differencing Scheme

The Central Differencing Scheme (CDS) interpolates the face value of ϕ using the arithmetic average between the two neighbors of that face.

$$\phi_e = \frac{\phi_E + \phi_P}{2} \tag{3.1}$$

$$\phi_w = \frac{\phi_W + \phi_P}{2} \tag{3.2}$$

3.1.1 Internal Cells

Substituting both (3.1) and (3.2) into Equation (2.6) gives:

$$F\frac{\phi_E + \phi_P}{2} - F\frac{\phi_W + \phi_P}{2} = D(\phi_E - \phi_P) + D(\phi_W - \phi_P) \tag{3.3}$$

After rearranging this equation into the format of $a_p\phi_P = a_w\phi_W + a_e\phi_E + S$:

$$2D\phi_P = (D + \frac{F}{2})\phi_W + (D - \frac{F}{2})\phi_E \tag{3.4}$$

In terms of P_e:

$$2\phi_P = (1 + \frac{P_e}{2})\phi_W + (1 - \frac{P_e}{2})\phi_E \tag{3.5}$$

3.1.2 Boundary Cells

In CDS the boundary faces take their exact boundary value since they are Dirichlet conditions. The equations for the first and last control volumes are:

$$3\phi_P = (2 + P_e)\phi_L + (1 - \frac{P_e}{2})\phi_E \tag{3.6}$$

and, for the last cell:

$$3\phi_P = (1 + \frac{P_e}{2})\phi_W + (2 - P_e)\phi_R \tag{3.7}$$

3.2 Upwind Differencing Scheme

3.2.1 Internal Cells

The Upwind Differencing Scheme (UDS) is a first-order scheme that evaluates the face value with the upstream value. For a positive flow $u > 0,\ w \to e$:

$$\phi_e = \phi_P \tag{3.8}$$

$$\phi_w = \phi_W \tag{3.9}$$

Substituting those two equations in Equation (2.6):

$$F\phi_P - F\phi_W = D(\phi_E - \phi_P) + D(\phi_W - \phi_P) \tag{3.10}$$

After rearranging:

$$(F + 2D)\phi_P = (D + F)\phi_W + D\phi_E \tag{3.11}$$

$$(2 + P_e)\phi_P = (1 + P_e)\phi_W + \phi_E \tag{3.12}$$

3.2.2 Boundary Cells

The same principle is applied on the boundary faces. For the first cell:

$$(3 + P_e)\phi_P = (2 + P_e)\phi_L + \phi_E \tag{3.13}$$

and for the last cell:

$$(3 + P_e)\phi_P = (1 + P_e)\phi_W + 2\phi_R \tag{3.14}$$

3.3 Hybrid Scheme

The Hybrid Scheme switches between two regimes depending on the local Péclet number, according to the stability limit of the Central Differencing Scheme:

$$|P_e| < 2: \quad \text{Central Differencing (full diffusion and convection)} \tag{3.15}$$

$$|P_e| \geq 2: \quad \text{diffusion is neglected and pure upwind convection is used} \tag{3.16}$$

3.3.1 Internal Cells

For $|P_e| < 2$, the equation is identical to CDS:

$$2D\phi_P = (D + \frac{F}{2})\phi_W + (D - \frac{F}{2})\phi_E \tag{3.17}$$

$$2\phi_P = (1 + \frac{P_e}{2})\phi_W + (1 - \frac{P_e}{2})\phi_E \tag{3.18}$$

For $|P_e| \geq 2$, the diffusive terms are dropped ($D = 0$) from Equation (2.6), and the convective face values are evaluated with UDS, i.e. for $u > 0,\ w \to e$: $\phi_e = \phi_P$ and $\phi_w = \phi_W$:

$$F\phi_P - F\phi_W = 0 \tag{3.19}$$

$$\phi_P = \phi_W \tag{3.20}$$

3.3.2 Boundary Cells

First Cell

For $|P_e| < 2$, the equation is identical to CDS:

$$3\phi_P = (2 + P_e)\phi_L + (1 - \frac{P_e}{2})\phi_E \tag{3.21}$$

For $|P_e| \geq 2$, the diffusive terms are dropped from Equation (2.9), with $\phi_e = \phi_P$ and $\phi_w = \phi_L$:

$$F\phi_P - F\phi_L = 0 \tag{3.22}$$

$$\phi_P = \phi_L \tag{3.23}$$

Last Cell

For $|P_e| < 2$, the equation is identical to CDS:

$$3\phi_P = (1 + \frac{P_e}{2})\phi_W + (2 - P_e)\phi_R \tag{3.24}$$

For $|P_e| \geq 2$, the diffusive terms are dropped from Equation (2.10), with $\phi_e = \phi_P$ and $\phi_w = \phi_W$:

$$F\phi_P - F\phi_W = 0 \tag{3.25}$$

$$\phi_P = \phi_W \tag{3.26}$$

3.4 Second-Order Upwind

3.4.1 Internal Cells

Second-Order Upwind (SOU) uses two upstream points instead of only one, by establishing a linear gradient and extrapolating the face value.

$$\phi_e = \phi_P + \nabla_e(\phi)\frac{\delta x}{2} \tag{3.27}$$

$$\phi_w = \phi_W + \nabla_w(\phi)\frac{\delta x}{2} \tag{3.28}$$

The gradient of ϕ is calculated using two upstream points. Based on the mesh shown in Figure 3.1, the gradient is calculated by:

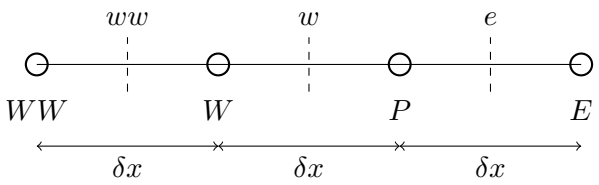


Figure 3.1: One-dimensional control volume for SOU interpolation.

$$\nabla_e(\phi) = \frac{\phi_P - \phi_W}{\delta x} \tag{3.29}$$

$$\nabla_w(\phi) = \frac{\phi_W - \phi_{WW}}{\delta x} \tag{3.30}$$

Substituting Equation (3.29) in Equation (3.27) gives:

$$\phi_e = \frac{3}{2}\phi_P - \frac{1}{2}\phi_W \tag{3.31}$$

and Equation (3.30) in Equation (3.28):

$$\phi_w = \frac{3}{2}\phi_W - \frac{1}{2}\phi_{WW} \tag{3.32}$$

The final equation for the internal cells is:

$$(2D + \frac{3}{2}F)\phi_P = -\frac{1}{2}F\phi_{WW} + (D + 2F)\phi_W + D\phi_E \tag{3.33}$$

$$(2 + \frac{3}{2}P_e)\phi_P = -\frac{1}{2}P_e\phi_{WW} + (1 + 2P_e)\phi_W + \phi_E \tag{3.34}$$

3.4.2 Boundary Cells

For the boundary cells, the gradient will be changed based on the upstream nodes.

First Cell

For the first cell, the west face is a Dirichlet boundary which will take the exact value ϕ_L, and the east face is extrapolated from ϕ_P and ϕ_L.

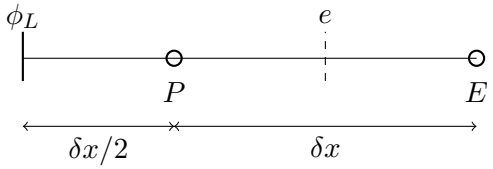


Figure 3.2: First control volume for SOU interpolation.

$$\phi_w = \phi_L \tag{3.35}$$

$$\nabla_e(\phi) = \frac{\phi_P - \phi_L}{\frac{\delta x}{2}} \tag{3.36}$$

$$\phi_e = 2\phi_P - \phi_L \tag{3.37}$$

The final equation is:

$$(3D + 2F)\phi_P = (2D + 2F)\phi_L + D\phi_E \tag{3.38}$$

$$(3 + 2P_e)\phi_P = (2 + 2P_e)\phi_L + \phi_E \tag{3.39}$$

Second Cell

The east face's gradient of the second cell is the same as the internal cells:

$$\nabla_e(\phi) = \frac{\phi_P - \phi_w}{\delta x} \tag{3.40}$$

$$\phi_e = \frac{3}{2}\phi_P - \frac{1}{2}\phi_W \tag{3.41}$$

while the west face has the following gradient:

$$\nabla_w(\phi) = \frac{\phi_W - \phi_L}{\frac{\delta x}{2}} \tag{3.42}$$

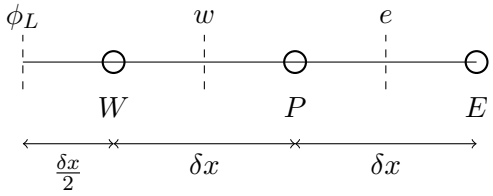


Figure 3.3: Second control volume for SOU interpolation.

$$\phi_w = 2\phi_W - \phi_L \tag{3.43}$$

The final equation is:

$$(2D + \frac{3}{2}F)\phi_P = -F\phi_L + (D + \frac{5}{2}F)\phi_W + D\phi_E \tag{3.44}$$

$$(2 + \frac{3}{2}P_e)\phi_P = -P_e\phi_L + (1 + \frac{5}{2}P_e)\phi_W + \phi_E \tag{3.45}$$

Last Cell

The last cell has the same treatment as the internal cells; the east face will have an extrapolated value instead of an exact value.

$$\phi_e = \frac{3}{2}\phi_P - \frac{1}{2}\phi_W \tag{3.46}$$

$$\phi_w = \frac{3}{2}\phi_W - \frac{1}{2}\phi_{WW} \tag{3.47}$$

The final equation is:

$$(3D + \frac{3}{2}F)\phi_P = -\frac{1}{2}F\phi_{WW} + (D + 2F)\phi_W + 2D\phi_R \tag{3.48}$$

$$(3 + \frac{3}{2}P_e)\phi_P = -\frac{1}{2}P_e\phi_{WW} + (1 + 2P_e)\phi_W + 2\phi_R \tag{3.49}$$

3.5 Quadratic Upstream Interpolation for Convective Kinetics

Quadratic Upstream Interpolation for Convective Kinetics (QUICK) interpolates the face value using two upstream nodes and one downstream node, i.e., the west face is interpolated with ϕ_{WW}, ϕ_W and ϕ_P, and the east face is interpolated with ϕ_W, ϕ_P and ϕ_E. The interpolation is a quadratic interpolation, i.e.,

$$\phi(x) = ax^2 + bx + c \tag{3.50}$$

where the coefficients a, b and c are determined.

3.5.1 Internal Cells

For the west face, applying the following conditions:

$$\phi(0) = \phi_{WW}$$

$$\phi(\delta x) = \phi_W$$

$$\phi(2\delta x) = \phi_P$$

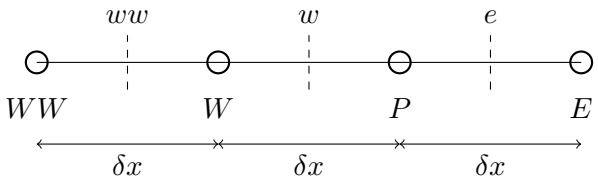


Figure 3.4: One-dimensional control volume for QUICK interpolation.

yields the following function for ϕ:

$$\phi(x) = \frac{\phi_P + \phi_{WW} - 2\phi_W}{2\delta x^2}x^2 + \frac{4\phi_W - 3\phi_{WW} - \phi_P}{2\delta x}x + \phi_{WW} \tag{3.51}$$

Then, the face value can be calculated as:

$$\phi_w = \phi(\frac{3}{2}\delta x) = \frac{6}{8}\phi_W + \frac{3}{8}\phi_P - \frac{1}{8}\phi_{WW} \tag{3.52}$$

The same steps are applied to the east face, yielding:

$$\phi_e = \phi(\frac{3}{2}\delta x) = \frac{6}{8}\phi_P + \frac{3}{8}\phi_E - \frac{1}{8}\phi_W \tag{3.53}$$

Substituting both in Equation (2.6) gives:

$$(2D + \frac{3}{8}F)\phi_P = -\frac{1}{8}F\phi_{WW} + (D + \frac{7}{8}F)\phi_W + (D - \frac{3}{8}F)\phi_E \tag{3.54}$$

$$(2 + \frac{3}{8}P_e)\phi_P = -\frac{1}{8}P_e\phi_{WW} + (1 + \frac{7}{8}P_e)\phi_W + (1 - \frac{3}{8}P_e)\phi_E \tag{3.55}$$

3.5.2 Boundary Cells

First Cell

In the case of the first cell, the west face takes the exact value ϕ_L, and the east face will be interpolated. From Figure 3.5, the interpolation conditions are:

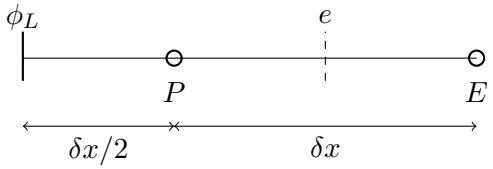


Figure 3.5: First control volume for QUICK interpolation.

$$\phi(0) = \phi_L$$

$$\phi(\frac{1}{2}\delta x) = \phi_P$$

$$\phi(\frac{3}{2}\delta x) = \phi_E$$

The function of ϕ becomes:

$$\phi(x) = \frac{-12\phi_P + 4\phi_E + 8\phi_L}{6\delta x^2}x^2 + \frac{9\phi_P - \phi_E - 8\phi_L}{3\delta x}x + \phi_L \tag{3.56}$$

and the east face is:

$$\phi_e = \phi(\delta x) = \phi_P - \frac{1}{3}\phi_L + \frac{1}{3}\phi_E \tag{3.57}$$

The final equation will be:

$$(3D + F)\phi_P = (2D + \frac{4}{3}F)\phi_L + (D - \frac{1}{3}F)\phi_E \tag{3.58}$$

$$(3 + P_e)\phi_P = (2 + \frac{4}{3}P_e)\phi_L + (1 - \frac{1}{3}P_e)\phi_E \tag{3.59}$$

Second Cell

In summary, the face values of the second cell are:

$$\phi_w = \phi(\frac{1}{2}\delta x) = \phi_W - \frac{1}{3}\phi_P + \frac{1}{3}\phi_L \tag{3.60}$$

$$\phi_e = \phi(\frac{3}{2}\delta x) = \frac{6}{8}\phi_P + \frac{3}{8}\phi_E - \frac{1}{8}\phi_W \tag{3.61}$$

and the final equation is:

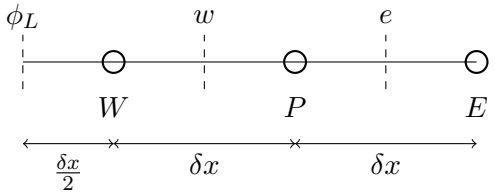


Figure 3.6: Second control volume for QUICK interpolation.

$$(2D + \frac{5}{12}F)\phi_P = -\frac{1}{3}F\phi_L + (D + \frac{9}{8}F)\phi_W + (D - \frac{3}{8}F)\phi_E \tag{3.62}$$

Last Cell

The last cell requires special treatment: the west face is treated the same way as in the internal cells, but the east face is a special case, sinceno downstream cell is available for interpolation. In this case, the east face value is extrapolated using two upstream nodes, which means applying SOU.

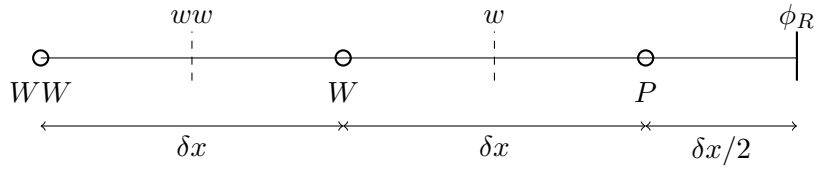


Figure 3.7: Last control volume for QUICK interpolation.

$$\phi_w = \phi(\frac{3}{2}\delta x) = \frac{6}{8}\phi_P + \frac{3}{8}\phi_E - \frac{1}{8}\phi_W \tag{3.63}$$

$$\phi_e = \frac{3}{2}\phi_P - \frac{1}{2}\phi_W \tag{3.64}$$

The final equation will be:

$$(3D + \frac{9}{8}F)\phi_P = -\frac{1}{8}F\phi_{WW} + (D + \frac{10}{8}F)\phi_W + 2D\phi_R \tag{3.65}$$

$$(3 + \frac{9}{8}P_e)\phi_P = -\frac{1}{8}P_e\phi_{WW} + (1 + \frac{10}{8}P_e)\phi_W + 2\phi_R \tag{3.66}$$

3.6 Exponential Scheme

The Exponential Scheme uses the exact solution of Section 2.2, applied locally between the two nodes surrounding a face.

3.6.1 Internal Cells

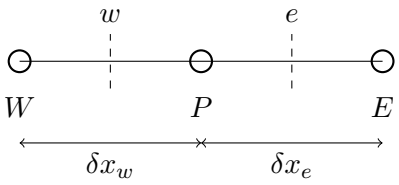


Figure 3.8: One-dimensional control volume for the exponential scheme.

Consider the face e between P and E, separated by δx, and apply Equation (2.21) locally on this interval with $\phi(0) = \phi_P$, $\phi(\delta x) = \phi_E$ and local Péclet number $P_e = \frac{F\delta x}{\Gamma}$. Using Equation (2.16) and Equation (2.17) for this interval:

$$\phi_P = C + \frac{J_e}{F} \qquad \phi_E = Ce^{P_e} + \frac{J_e}{F} \tag{3.67}$$

Subtracting gives C:

$$C = \frac{\phi_P - \phi_E}{1 - e^{P_e}} \tag{3.68}$$

and substituting back gives the flux at the east face:

$$J_e = F(\phi_P - C) = F\phi_P - \frac{F(\phi_P - \phi_E)}{1 - e^{P_e}} \tag{3.69}$$

$$J_e = F\phi_P + \frac{F}{e^{P_e} - 1}(\phi_P - \phi_E) \tag{3.70}$$

Applying the same procedure on the interval between W and P with $\phi(0) = \phi_W$, $\phi(\delta x) = \phi_P$:

$$J_w = F\phi_W + \frac{F}{e^{P_e} - 1}(\phi_W - \phi_P) \tag{3.71}$$

The discrete balance for the internal cell is $J_e - J_w = 0$. Substituting by both equations (3.70) and (3.71):

$$F\phi_P + \frac{F}{e^{P_e} - 1}(\phi_P - \phi_E) - F\phi_W - \frac{F}{e^{P_e} - 1}(\phi_W - \phi_P) = 0 \tag{3.72}$$

Rearranging into $a_P\phi_P = a_W\phi_W + a_E\phi_E$ form:

$$(1 + \frac{2}{e^{P_e} - 1})F\phi_P = \frac{F}{e^{P_e} - 1}\phi_E + (1 + \frac{1}{e^{P_e} - 1})F\phi_W \tag{3.73}$$

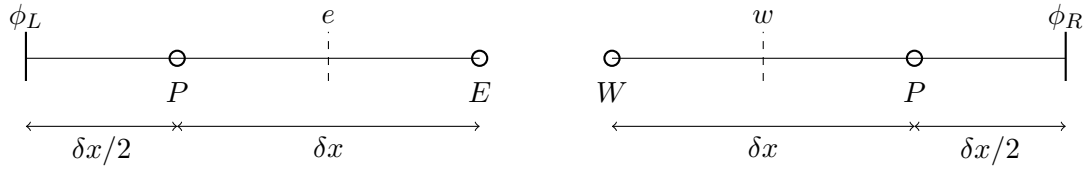
3.6.2 Boundary Cells

First Cell

For the first cell, the west face is Dirichlet and takes the exact value ϕ_L, but in order to apply the exponential scheme conservation of flux should be applied.

Applying Equation (2.21) between P and L, with $\phi(0) = \phi_L$, $\phi(\delta x/2) = \phi_P$ and local Péclet number $P_e = \frac{F\delta x}{\Gamma}$:

$$\phi_L = C + \frac{J_w}{F} \qquad \phi_P = Ce^{P_e/2} + \frac{J_e}{F} \tag{3.74}$$



(a) First cell (west boundary)

(b) Last cell (east boundary)

Figure 3.9: Boundary control volumes for the exponential scheme.

$$C = \frac{\phi_L - \phi_P}{1 - e^{P_e/2}} \tag{3.75}$$

$$J_w = F\phi_L - \frac{F(\phi_L - \phi_P)}{1 - e^{P_e/2}} \tag{3.76}$$

The discrete balance for the last cell is $J_e - J_w = 0$, with J_e as given by Equation (3.70). by Substituting and rearranging:

$$(1 + \frac{1}{e^{P_e} - 1} + \frac{1}{e^{P_e/2} - 1})F\phi_P = (1 + \frac{1}{e^{P_e} - 1})F\phi_L + \frac{F}{e^{P_e/2} - 1}\phi_E \tag{3.77}$$

$$(1 + \frac{1}{e^{P_e} - 1} + \frac{1}{e^{P_e/2} - 1})P_e\phi_P = (1 + \frac{1}{e^{P_e} - 1})P_e\phi_L + \frac{Pe}{e^{P_e/2} - 1}\phi_E \tag{3.78}$$

Last Cell

Applying Equation (2.21) between P and L, with $\phi(0) = \phi_P$, $\phi(\delta x/2) = \phi_R$ and local Péclet number $P_e = \frac{F\delta x}{\Gamma}$:

$$\phi_P = C + \frac{J_w}{F} \qquad \phi_R = Ce^{P_e/2} + \frac{J_e}{F} \tag{3.79}$$

$$C = \frac{\phi_P - \phi_R}{1 - e^{P_e/2}} \tag{3.80}$$

$$J_w = F\phi_P - \frac{F(\phi_P - \phi_R)}{1 - e^{P_e/2}} \tag{3.81}$$

The discrete balance for the last cell is $J_e - J_w = 0$, with J_w as given by Equation (3.71). by Substituting and rearranging:

$$(1 + \frac{1}{e^{P_e} - 1} + \frac{1}{e^{P_e/2} - 1})F\phi_P = (1 + \frac{1}{e^{P_e} - 1})F\phi_P + \frac{F}{e^{P_e/2} - 1}\phi_R \tag{3.82}$$

$$(1 + \frac{1}{e^{P_e} - 1} + \frac{1}{e^{P_e/2} - 1})P_e\phi_P = (1 + \frac{1}{e^{P_e} - 1})P_e\phi_P + \frac{Pe}{e^{P_e/2} - 1}\phi_R \tag{3.83}$$

3.7 Power Law Scheme

The Power Law Scheme is built directly from the Upwind Differencing Scheme: the convective face values are taken exactly as in UDS, but the diffusive conductance D is scaled by a function f that mimics the behavior of the exact exponential solution of Section 2.2 while remaining cheap to evaluate. The function is defined as:

$$f = (1 - 0.1P_e)^5 \tag{3.84}$$

3.7.1 Internal Cells

As in UDS, for a positive flow $u > 0,\ w \to e$:

$$\phi_e = \phi_P \tag{3.85}$$

$$\phi_w = \phi_W \tag{3.86}$$

Substituting the diffusive conductance D with Df and substituting into Equation (2.6):

$$F\phi_P - F\phi_W = Df(\phi_E - \phi_P) + Df(\phi_W - \phi_P) \tag{3.87}$$

After rearranging into the format of $a_P\phi_P = a_W\phi_W + a_E\phi_E$:

$$(F + 2Df)\phi_P = (Df + F)\phi_W + Df\phi_E \tag{3.88}$$

In terms of P_e:

$$(P_e + 2f)\phi_P = (f + P_e)\phi_W + f\phi_E \tag{3.89}$$

with f given by Equation (3.84).

3.7.2 Boundary Cells

The same substitution $D \to Df$ is applied to the boundary treatment. For the first cell, following Equation (2.9) with $\phi_e = \phi_P$ and $\phi_w = \phi_L$:

$$F\phi_P - F\phi_L = Df(\phi_E - \phi_P) + 2Df(\phi_L - \phi_P) \tag{3.90}$$

$$(F + 3Df)\phi_P = (2Df + F)\phi_L + Df\phi_E \tag{3.91}$$

$$(P_e + 3f)\phi_P = (2f + P_e)\phi_L + f\phi_E \tag{3.92}$$

For the last cell, following Equation (2.10) with $\phi_e = \phi_P$ and $\phi_w = \phi_W$:

$$F\phi_P - F\phi_W = 2Df(\phi_R - \phi_P) + Df(\phi_W - \phi_P) \tag{3.93}$$

$$(F + 3Df)\phi_P = (Df + F)\phi_W + 2Df\phi_R \tag{3.94}$$

$$(P_e + 3f)\phi_P = (f + P_e)\phi_W + 2f\phi_R \tag{3.95}$$

3.8 Monotonic Upstream-Centered Scheme for Conservation Laws

The Monotonic Upstream-Centered Scheme for Conservation Laws (MUSCL) is a non-linear, gradient-limited extension of SOU. The face value is written as the upstream value plus a limited correction based on the SOU gradient:

$$\phi_e = \phi_P + \frac{1}{2}\psi(r_e)(\phi_P - \phi_W) \tag{3.96}$$

$$\phi_w = \phi_W + \frac{1}{2}\psi(r_w)(\phi_W - \phi_{WW}) \tag{3.97}$$

where ψ is the Van Leer limiter function:

$$\psi(r) = \frac{r + |r|}{1 + |r|} \tag{3.98}$$

and r is the ratio of consecutive gradients:

$$r_e = \frac{\phi_P - \phi_W}{\phi_E - \phi_P}, \qquad r_w = \frac{\phi_W - \phi_{WW}}{\phi_P - \phi_W} \tag{3.99}$$

Because ψ depends non-linearly on ϕ, $\psi_e = \psi(r_e)$ and $\psi_w = \psi(r_w)$ are evaluated from the previous iteration's ϕ field and treated as fixed coefficients when assembling the current iteration's linear system, so the equation below retains the standard $a_P\phi_P = a_W\phi_W + a_E\phi_E$ form.

3.8.1 Internal Cells

Substituting equations (3.96) and (3.97) into Equation (2.6):

$$F\left[\phi_P + \frac{\psi_e}{2}(\phi_P - \phi_W)\right] - F\left[\phi_W + \frac{\psi_w}{2}(\phi_W - \phi_{WW})\right] = D(\phi_E - \phi_P) + D(\phi_W - \phi_P) \tag{3.100}$$

After rearranging into the format of $a_P\phi_P = a_{WW}\phi_{WW} + a_W\phi_W + a_E\phi_E$:

$$\left(2D + F + \frac{F\psi_e}{2}\right)\phi_P = -\frac{F\psi_w}{2}\phi_{WW} + \left(D + F + \frac{F\psi_w}{2} + \frac{F\psi_e}{2}\right)\phi_W + D\phi_E \tag{3.101}$$

In terms of P_e:

$$\left(2 + P_e + \frac{P_e\psi_e}{2}\right)\phi_P = -\frac{P_e\psi_w}{2}\phi_{WW} + \left(1 + P_e + \frac{P_e\psi_w}{2} + \frac{P_e\psi_e}{2}\right)\phi_W + \phi_E \tag{3.102}$$

3.8.2 Boundary Cells

First Cell

The west face is Dirichlet, $\phi_w = \phi_L$. The east face uses the boundary gradient ratio, formed using the half-distance to ϕ_L:

$$r_e = \frac{2(\phi_P - \phi_L)}{\phi_E - \phi_P} \tag{3.103}$$

$$\phi_e = \phi_P + \psi(r_e)(\phi_P - \phi_L) \tag{3.104}$$

Substituting into Equation (2.9):

$$(3 + P_e + P_e\psi_e)\phi_P = (2 + P_e + P_e\psi_e)\phi_L + \phi_E \tag{3.105}$$

Second Cell

The east face follows the internal-cell treatment:

$$\phi_e = \phi_P + \frac{\psi_e}{2}(\phi_P - \phi_W), \qquad r_e = \frac{\phi_P - \phi_W}{\phi_E - \phi_P} \tag{3.106}$$

The west face uses the boundary gradient ratio, formed from the half-distance to ϕ_L:

$$r_w = \frac{2(\phi_W - \phi_L)}{\phi_P - \phi_W} \tag{3.107}$$

$$\phi_w = \phi_W + \psi(r_w)(\phi_W - \phi_L) \tag{3.108}$$

Substituting into Equation (2.6):

$$\left(2 + P_e + \frac{P_e\psi_e}{2}\right)\phi_P = -P_e\psi_w\phi_L + \left(1 + P_e + P_e\psi_w - \frac{P_e\psi_e}{2}\right)\phi_W + \phi_E \tag{3.109}$$

Last Cell

The west face follows the internal-cell treatment:

$$\phi_w = \phi_W + \frac{\psi_w}{2}(\phi_W - \phi_{WW}), \qquad r_w = \frac{\phi_W - \phi_{WW}}{\phi_P - \phi_W} \tag{3.110}$$

The east face uses the boundary gradient ratio, formed from the half-distance to ϕ_R:

$$r_e = \frac{\phi_P - \phi_W}{2(\phi_R - \phi_P)} \tag{3.111}$$

$$\phi_e = \phi_P + \frac{1}{2}\psi(r_e)(\phi_P - \phi_W) \tag{3.112}$$

Substituting into Equation (2.10):

$$\left(3 + P_e + \frac{P_e\psi_e}{2}\right)\phi_P = -\frac{P_e\psi_w}{2}\phi_{WW} + \left(1 + P_e + \frac{P_e\psi_w}{2} - \frac{P_e\psi_e}{2}\right)\phi_W + 2\phi_R \tag{3.113}$$

Chapter 4

VBA Implementation

4.1 Overview

The solver is organized into four layers, each with a single responsibility. The `Cases` module defines and launches a run: it specifies the scheme to use, the Péclet number or numbers to solve for, the worksheet interval holding the cells to work on, and the maximum number of iterations allowed. Cells are laid out as rows on the worksheet, with each row holding one complete set of ϕ values for a given Péclet number; a case can therefore define a single Péclet number to solve a single row, or an array of several Péclet numbers to solve several rows in the same run, depending on what the user wants to compare.

Once a case is defined, it calls `Run`, performs the execution of the program with the parameters already set: it reads the worksheet, calls the appropriate scheme module, runs the solver, and writes the results and residuals back to the worksheet, all from the same call.

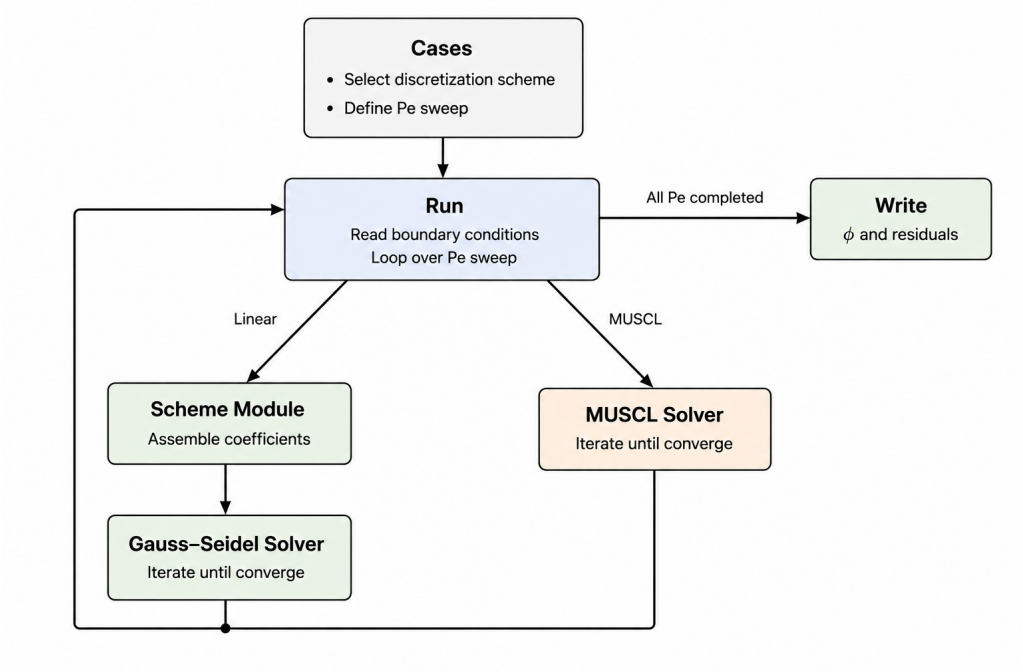


Figure 4.1: Architecture and execution flow of the VBA solver.

4.2 Case Orchestration

A case fixes a scheme, a mesh resolution, and a sweep of Péclet numbers, and calls the orchestration routine once per case. As an example, running the Central Differencing Scheme over a fixed set of Péclet numbers on a given worksheet range consists of specifying the scheme name, the worksheet interval to read and write, the Péclet number sweep, and the solver's stopping criteria:

```
Sub CDS_localPe()
    Dim max_iter As Long
    Dim residual_line As Integer

    Pe = Array(1, 2, 4, 10, 25, 50)
    interval = "D5:L15"
    Scheme = "CDS"
    max_iter = 10000
    residual_line = 3

    Run.Run Pe, Scheme, interval, max_iter, residual_line
End Sub
```

Two conventions are used across cases to define the Péclet number swept for a given mesh: a *local* convention, where each value in the sweep is the cell-level Péclet number itself, independent of the number of cells in the mesh; and a *global* convention, where each value represents a fixed domain-level Péclet number, divided by the number of cells to obtain the corresponding local (cell-level) Péclet number that is actually passed to the scheme and solver. The latter allows the same domain-level flow condition to be compared consistently across meshes of different resolution.

4.3 The Run Module

Once a case has defined its scheme, Péclet sweep, worksheet interval, and iteration cap, `Run` carries out the complete execution. It first reads the full worksheet interval into memory as a single array, where each Péclet number in the sweep occupies its own row.

For each Péclet number in turn, `Run` extracts that row into a separate array, selects and calls the matching scheme module to obtain its coefficients (or, for MUSCL, skips this step entirely and calls the dedicated MUSCL solver directly), then passes the row to the solver. Once the solver returns, the updated row is written back into the in-memory array at its original position, and the residual history for that Péclet number is written to the results sheet immediately, before moving on to the next Péclet number in the sweep. Only after every Péclet number in the sweep has been solved is the full array written back to the worksheet in a single operation:

```
Public Sub Run(Pe, Scheme, cells_interval, max_iter As Long,
               residual_line As Integer)

    Dim Phi As Variant
    Phi = Sheets("Study").Range(cells_interval).Value

    For i = LBound(Pe) To UBound(Pe)

        Dim residual_history() As Double
```

```
        Dim Phi_row() As Double
        ReDim Phi_row(1 To UBound(Phi, 2))
        For j = 1 To UBound(Phi, 2)
            Phi_row(j) = Phi(1 + 2 * i, j)
        Next j

        Dim C As Coefficients
        Select Case Scheme
            Case "MUSCL"
                residual_history = Solver.MUSCL_Solver(Phi_row, Pe(i),
                                                max_iter, 0.000001)
            Case Else
                Select Case Scheme
                    Case "CDS"
                        C = Schemes.CDS(Pe(i))
                    Case "UDS"
                        C = Schemes.UDS(Pe(i))
                    Case "Hybrid"
                        C = Schemes.Hybrid(Pe(i))
                    Case "SOU"
                        C = Schemes.SOU(Pe(i))
                    Case "QUICK"
                        C = Schemes.QUICK(Pe(i))
                    Case "Power_Law"
                        C = Schemes.Power_Law(Pe(i))
                    Case "Exp_S"
                        C = Schemes.Exp_S(Pe(i))
                End Select
                residual_history = Solver.Solver(Phi_row, C,
                                        max_iter, 0.000001)
        End Select

        For j = 1 To UBound(Phi, 2)
            Phi(1 + 2 * i, j) = Phi_row(j)
        Next j

        Write_residuals residual_history, residual_line + 4 * i

    Next i

    Sheets("Study").Range(cells_interval).Value = Phi

End Sub
```

The factor of two in `1 + 2*i` and the factor of four in `residual_line + 4*i` both came from worksheet layout for aesthetic purposes.

4.4 Coefficient Representation and Scheme Modules

Every linear scheme derived in Chapter 3 (CDS, UDS, Hybrid, SOU, QUICK, Power Law, Exponential) reduces to the same $a_P\phi_P = a_{WW}\phi_{WW} + a_W\phi_W + a_E\phi_E$ form, for each of the four cell types identified above. This makes it possible to represent any of these

seven schemes with a single structure, holding one ratio per neighbor per cell type, already normalized by a_P:

```
Public Type Coefficients
    r0_first As Double
    re_first As Double
    r0_second As Double
    rw_second As Double
    re_second As Double
    rww_intern As Double
    rw_intern As Double
    re_intern As Double
    rww_last As Double
    rw_last As Double
    r1_last As Double
End Type
```

A scheme module is a function that takes a Péclet number and returns this structure filled in, directly encoding the discretized equations derived in Chapter 2 for each cell type. Since the struct stores ratios rather than raw coefficients, each module computes a_P, a_W, a_E (and a_{WW} where relevant) exactly as derived, then divides through by a_P before storing the result.

As the simplest case, the UDS module builds its coefficients directly from the equations of Section 3.2:

```
Public Function UDS(Pe) As Coefficients

    Dim ap As Double
    Dim a0 As Double
    Dim aww As Double
    Dim aw As Double
    Dim ae As Double
    Dim a1 As Double
    Dim C As Coefficients

    ' First Cell
    ap = 3 + Pe
    a0 = 2 + Pe
    ae = 1

    C.r0_first = a0 / ap
    C.re_first = ae / ap

    ' Internal Cell
    ap = 2 + Pe
    aww = 0
    aw = 1 + Pe
    ae = 1

    C.rww_intern = aww / ap
    C.rw_intern = aw / ap
    C.re_intern = ae / ap
```

```
    ' Last Cell
    ap = 3 + Pe
    aww = 0
    aw = 1 + Pe
    a1 = 2

    C.rww_last = aww / ap
    C.rw_last = aw / ap
    C.r1_last = a1 / ap

    ' Second Cell
    ap = 2 + Pe
    a0 = 0
    aw = 1 + Pe
    ae = 1

    C.r0_second = a0 / ap
    C.rw_second = aw / ap
    C.re_second = ae / ap

    UDS = C
End Function
```

Each block computes the raw coefficients from Section 3.2's equations for one cell type, then stores them into `C` as ratios normalized by a_P, before moving to the next cell type.

4.5 The Generic Gauss-Seidel Solver

The solver sweeps the domain once per iteration, in a fixed order: first cell, second cell, all internal cells left to right, then the last cell. Each cell is updated immediately using its neighbors' latest values, so later cells in the same sweep already see the updated values of earlier ones.

```
Do While residual > tolerance And iter < max_iter

    ' First Cell
    Phi(start) = C.r0_first * Phi(start - 1) + C.re_first * Phi(start + 1)

    ' Second Cell
    Phi(start + 1) = C.r0_second * Phi(start - 1) _
                   + C.rw_second * Phi(start) _
                   + C.re_second * Phi(start + 2)

    ' Internal Cells
    For j = start + 2 To finish - 1
        Phi(j) = C.rww_intern * Phi(j - 2) _
               + C.rw_intern * Phi(j - 1) _
               + C.re_intern * Phi(j + 1)
    Next j

    ' Last Cell
```

```
    Phi(finish) = C.rww_last * Phi(finish - 2) _
                + C.rw_last * Phi(finish - 1) _
                + C.r1_last * Phi(finish + 1)

    iter = iter + 1

Loop
```

After every cell update, the relative change from the previous value is computed, and the largest such change across the whole sweep is taken as that iteration's residual. The sweep repeats until this residual falls below a fixed tolerance or a maximum iteration count is reached, whichever comes first; the full history of residuals is retained so that convergence behavior can be inspected afterward.

4.6 MUSCL: A Special Case

MUSCL cannot be represented with the same coefficient structure, since its face values depend on the Van Leer limiter ψ, which is itself a non-linear function of the local ϕ field. Freezing this dependency is what makes an otherwise non-linear scheme solvable with a linear sweep: at the start of each iteration, ψ is evaluated at every face using the ϕ field from the end of the previous iteration, and these values are then treated as fixed coefficients for the remainder of that iteration's sweep. This deferred-correction treatment means MUSCL uses its own solver routine, structurally identical to the generic one (same four cell types, same sweep order, same residual and stopping criteria) but recomputing its coefficients from the lagged field at the start of every pass, rather than precomputing them once from the Péclet number alone.

Chapter 5

Results and Validation Against OpenFOAM

5.1 OpenFOAM vs. Classical FVM

A direct comparison between the VBA solver and OpenFOAM is complicated by two differences in convention: how the outflow boundary is treated, and how the gradient is estimated for internal cells.

At the outflow boundary, textbook formulations for upstream schemes let the outflow carry out whatever value already exists inside the domain there, rather than imposing a fixed value on it. OpenFOAM's default fixedValue boundary condition, by contrast, applies the specified value at every boundary face marked with it, including outflow faces.

Separately, for internal faces, OpenFOAM's default gradient scheme is central differencing based, whereas the classical treatment used throughout Chapter 3 for the upstream schemes estimates the gradient using only upstream information.

Neither OpenFOAM convention is incorrect on its own, but both differ from the textbook treatment for these reasons, making a direct cell-for-cell comparison with a textbook-style solver impossible without addressing them first. To remove this discrepancy, a set of custom OpenFOAM libraries was built:

- `patankarFixedValue`, a boundary condition that behaves as a standard fixed value everywhere except at outflow, where it lets the outflow value be carried from inside the domain instead.
- `upstream`, a gradient scheme that estimates the local gradient using only upstream information instead of the central differencing based gradient.
- `patankarLinearUpwind`, `patankarQUICK`, `patankarVanLeer`, textbook-consistent versions of several higher-order interpolation schemes, identical to their built-in OpenFOAM counterparts everywhere inside the domain, but extended consistently to the outflow boundary rather than dropping to lower accuracy there.

None of these libraries alter OpenFOAM's default behavior; they are only active where explicitly selected in the case setup, and are intended specifically to reproduce the classical textbook convention for validation purposes, not as a general-purpose replacement for OpenFOAM's own schemes.

5.2 OpenFOAM Setup

The validation case was run with `scalarTransportFoam`, solving the same 1D convection-diffusion problem as Chapter 2 for a passive scalar with no additional physics. The gradient scheme and the interpolation scheme for the scalar were both selected explicitly in the `fvSchemes` dictionary to use the custom libraries introduced above:

```
gradSchemes
{
    default         upstream;
}

divSchemes
{
    default         none;
    div(phi,T)      Gauss patankarLinearUpwind upstream;
}

laplacianSchemes
{
    default         Gauss linear corrected;
}
```

and the outflow patch in `0/T` was set to use `patankarFixedValue` in place of the standard `fixedValue` condition.

5.3 Results

With this setup in place, the results of both the VBA solver and OpenFOAM are showed in the following tables:

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9771	99.8399	99.4281	98.1930	94.4876	83.3714	50.0228
OpenFOAM	99.9771	99.8399	99.4281	98.1930	94.4875	83.3713	50.0228

Table 5.1: CDS, $P_e = 1$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000
OpenFOAM	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000	100.0000

Table 5.2: CDS, $P_e = 2$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0914	99.5430	101.1883	96.2523	111.0603	66.6362	199.9086
OpenFOAM	100.0910	99.5430	101.1880	96.2522	111.0600	66.6361	199.9090

Table 5.3: CDS, $P_e = 4$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	127.6458	44.7084	169.1145	-17.4946	262.4190	-157.4514	472.3542
OpenFOAM	127.6460	44.7076	169.1150	-17.4955	262.4190	-157.4520	472.3540

Table 5.4: CDS, $P_e = 10$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	406.9583	-313.7264	532.2948	-460.8605	705.0174	-663.6219	943.0417
OpenFOAM	406.9580	-313.7250	532.2900	-460.8550	705.0080	-663.6120	943.0270

Table 5.5: CDS, $P_e = 25$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	1008.6957	-960.1450	1172.7658	-1137.8876	1365.3202	-1346.4882	1591.3043
OpenFOAM	1008.6400	-960.0770	1172.6900	-1137.8000	1365.2200	-1346.3900	1591.1900

Table 5.6: CDS, $P_e = 50$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.6503	98.6013	96.5034	92.3075	83.9159	67.1327	33.5664
OpenFOAM	99.6503	98.6013	96.5034	92.3075	83.9158	67.1326	33.5663

Table 5.7: UDS, $P_e = 1$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9657	99.8284	99.4167	98.1817	94.4767	83.3618	50.0171
OpenFOAM	99.9657	99.8285	99.4168	98.1817	94.4767	83.3618	50.0170

Table 5.8: UDS, $P_e = 2$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9986	99.9900	99.9473	99.7340	98.6673	93.3339	66.6671
OpenFOAM	99.9986	99.9900	99.9474	99.7340	98.6673	93.3339	66.6671

Table 5.9: UDS, $P_e = 4$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	99.9999	99.9989	99.9875	99.8622	98.4848	83.3333
OpenFOAM	100.0000	99.9999	99.9989	99.9875	99.8623	98.4848	83.3333

Table 5.10: UDS, $P_e = 10$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	99.9996	99.9890	99.7151	92.5926
OpenFOAM	100.0000	100.0000	100.0000	99.9996	99.9890	99.7151	92.5926

Table 5.11: UDS, $P_e = 25$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.0000	99.9985	99.9246	96.1538
OpenFOAM	100.0000	100.0000	100.0000	100.0000	99.9985	99.9246	96.1538

Table 5.12: UDS, $P_e = 50$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.8588	99.2941	98.0236	95.1297	88.5302	73.4785	39.1490
OpenFOAM	99.8588	99.2941	98.0236	95.1297	88.5301	73.4784	39.1489

Table 5.13: SOU, $P_e = 1$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9927	99.9492	99.7894	99.1941	96.9724	88.6815	57.7391
OpenFOAM	99.9927	99.9492	99.7895	99.1941	96.9725	88.6815	57.7392

Table 5.14: SOU, $P_e = 2$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9998	99.9982	99.9872	99.9138	99.4218	96.1250	74.0313
OpenFOAM	99.9998	99.9982	99.9872	99.9138	99.4219	96.1250	74.0313

Table 5.15: SOU, $P_e = 4$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	99.9998	99.9969	99.9512	99.2354	88.0104
OpenFOAM	100.0000	100.0000	99.9998	99.9969	99.9512	99.2354	88.0104

Table 5.16: SOU, $P_e = 10$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	99.9999	99.9965	99.8662	94.8944
OpenFOAM	100.0000	100.0000	100.0000	99.9999	99.9965	99.8662	94.8944

Table 5.17: SOU, $P_e = 25$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.0000	99.9995	99.9655	97.3914
OpenFOAM	100.0000	100.0000	100.0000	100.0000	99.9995	99.9655	97.3914

Table 5.18: SOU, $P_e = 50$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9499	99.6994	99.0080	97.1224	91.9809	77.9620	39.7374
OpenFOAM	99.9245	99.5721	98.6828	96.4289	90.7148	76.2288	39.5043

Table 5.19: QUICK, $P_e = 1$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	99.9996	99.9958	99.9579	99.5834	95.8759	59.1752
OpenFOAM	99.9995	99.9944	99.9646	99.7913	98.7811	92.8936	58.5789

Table 5.20: QUICK, $P_e = 2$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0007	99.9944	100.0453	99.6324	102.9857	75.7464
OpenFOAM	100.0000	100.0000	100.0000	100.0000	99.9999	99.9999	74.9999

Table 5.21: QUICK, $P_e = 4$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.4400	99.0600	103.4300	89.3000
OpenFOAM	100.0000	100.0000	99.9965	100.0260	99.8016	101.4990	88.6762

Table 5.22: QUICK, $P_e = 10$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.4300	99.3900	101.8300	95.5300
OpenFOAM	99.9998	100.0010	99.9934	100.0340	99.8223	100.9210	95.2281

Table 5.23: QUICK, $P_e = 25$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.2700	99.6600	101.0600	97.8000
OpenFOAM	99.9998	100.0010	99.9949	100.0240	99.8885	100.5200	97.5715

Table 5.24: QUICK, $P_e = 50$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.8042	99.0797	97.5364	94.1939	86.9420	71.2051	37.0547
OpenFOAM	99.7076	98.6948	96.7382	92.8584	85.1154	69.6378	38.6868

Table 5.25: MUSCL, $P_e = 1$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9841	99.9024	99.6368	98.7555	95.8269	86.0942	53.7487
OpenFOAM	99.9706	99.8530	99.5001	98.4416	95.2660	85.7394	57.1595

Table 5.26: MUSCL, $P_e = 2$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	99.9993	99.9943	99.9668	99.8158	98.9849	94.4121	69.2466
OpenFOAM	99.9989	99.9921	99.9584	99.7900	98.9479	94.7373	73.6846

Table 5.27: MUSCL, $P_e = 4$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	99.9999	99.9992	99.9902	99.8855	98.6582	84.2756
OpenFOAM	100.0000	99.9999	99.9992	99.9909	99.9001	98.9011	87.9121

Table 5.28: MUSCL, $P_e = 10$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	99.9996	99.9901	99.7328	92.8222
OpenFOAM	100.0000	100.0000	100.0000	99.9997	99.9924	99.8029	94.8743

Table 5.29: MUSCL, $P_e = 25$

	ϕ_1	ϕ_2	ϕ_3	ϕ_4	ϕ_5	ϕ_6	ϕ_7
VBA	100.0000	100.0000	100.0000	100.0000	99.9986	99.9272	96.2212
OpenFOAM	100.0000	100.0000	100.0000	100.0000	99.9990	99.9487	97.3860

Table 5.30: MUSCL, $P_e = 50$

5.4 Comparison

Comparison between the VBA solver, OpenFOAM, and exact solution scheme by scheme, across a sweep of Péclet numbers is showen in the following figures.

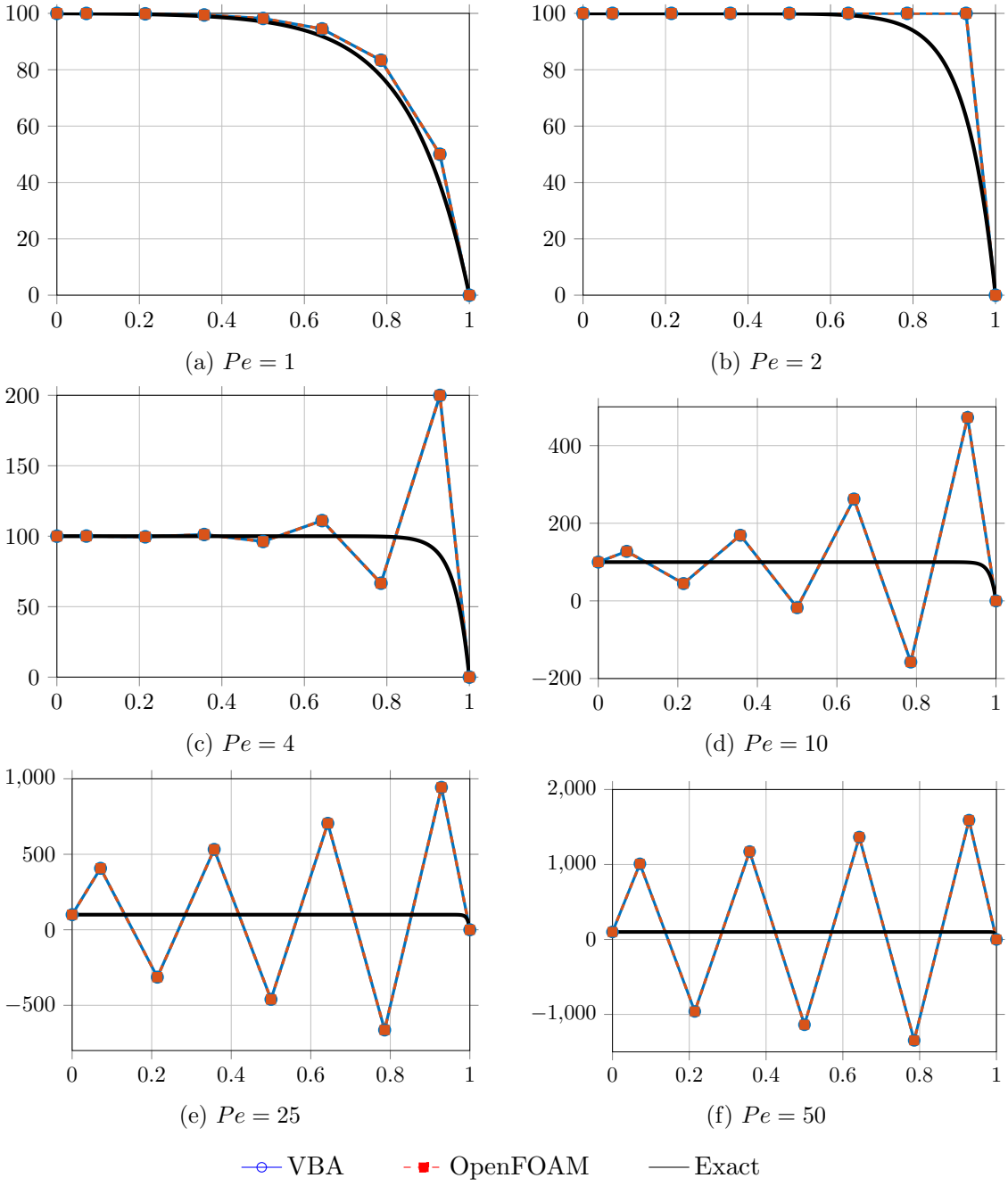


Figure 5.1: Comparison of VBA, OpenFOAM, and exact solutions for the Central Differencing Scheme (CDS).

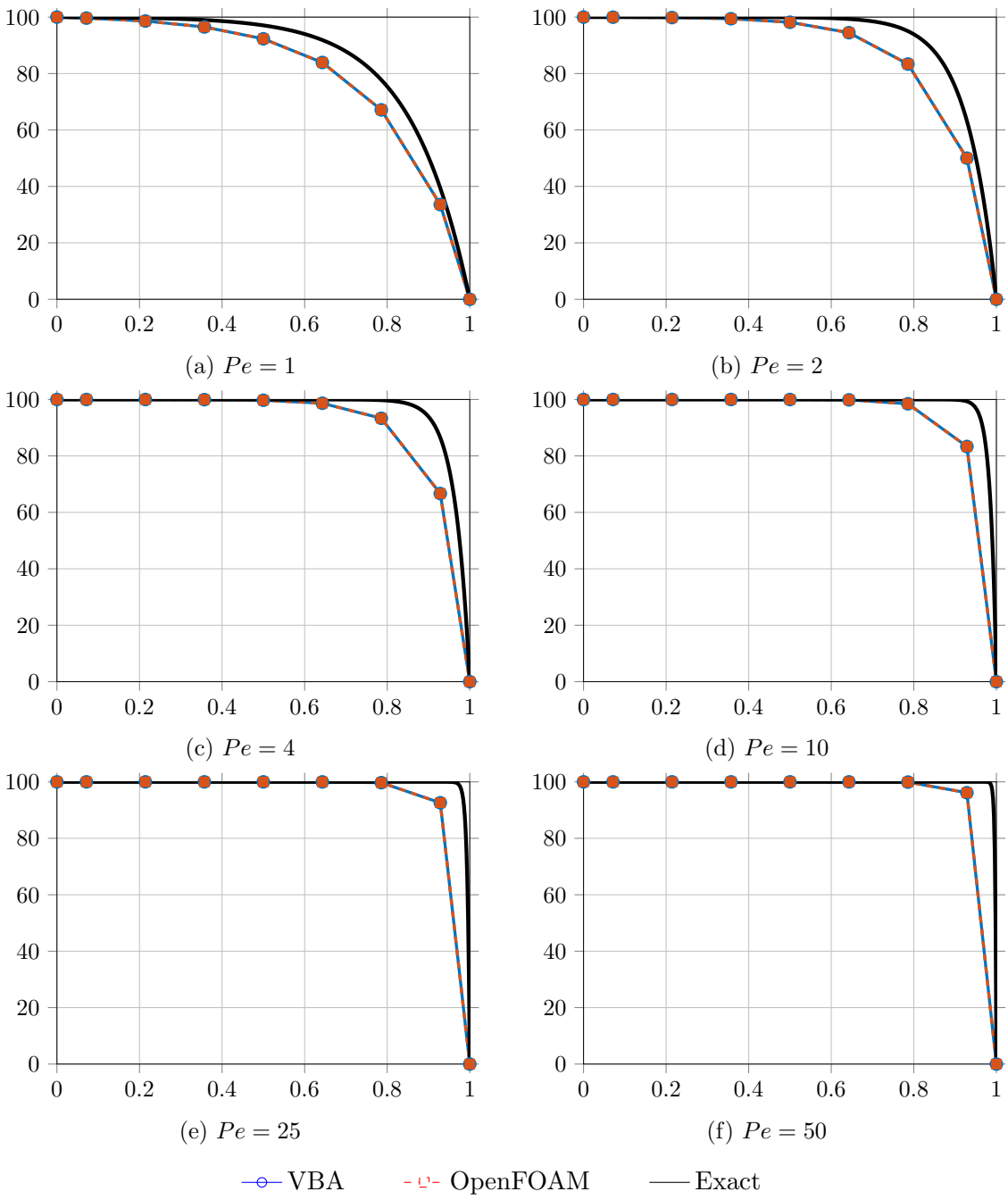


Figure 5.2: Comparison of VBA, OpenFOAM, and exact solutions for the Upwind Differencing Scheme (UDS).

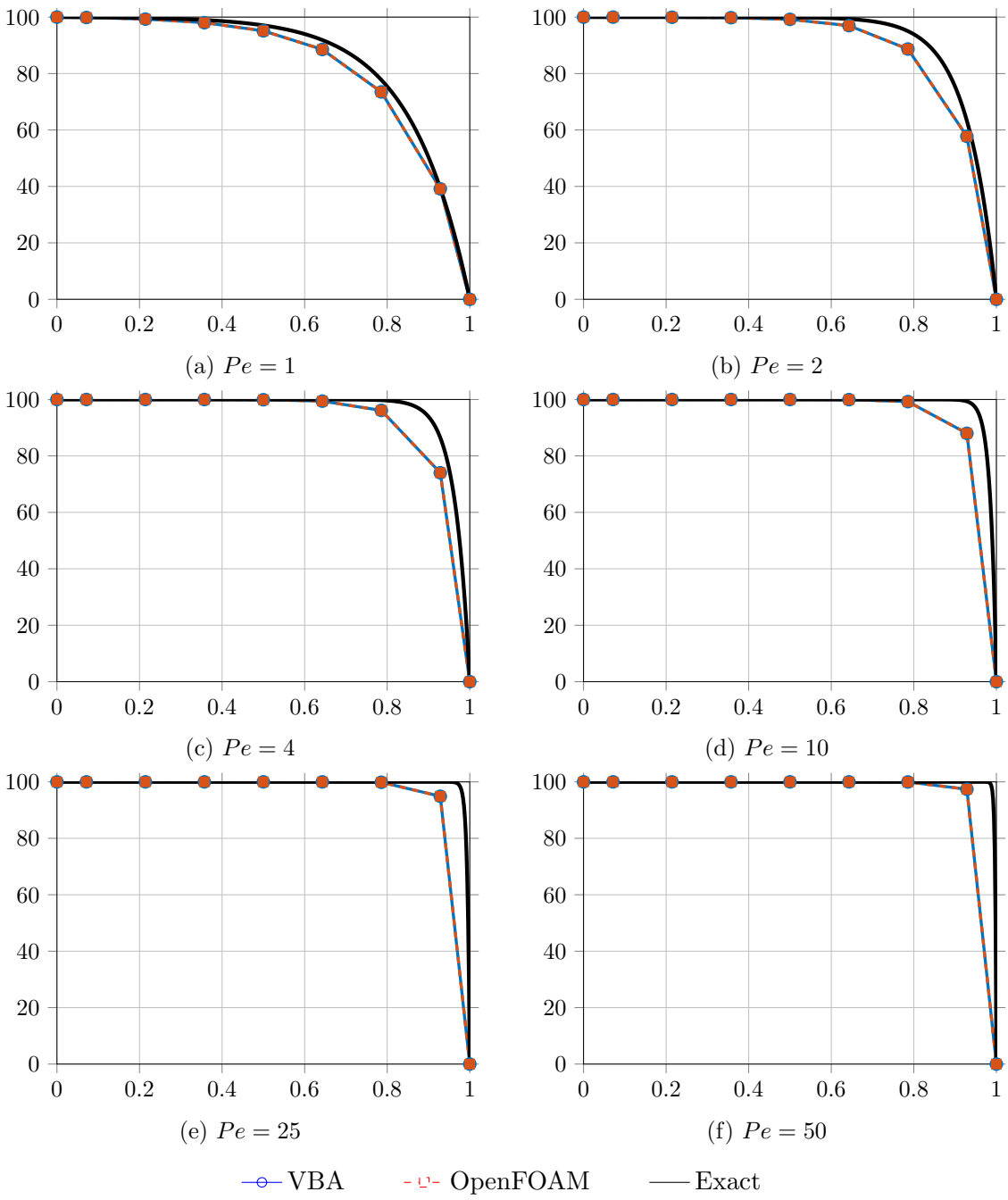


Figure 5.3: Comparison of VBA, OpenFOAM, and exact solutions for the Second-Order Upwind (SOU) scheme.

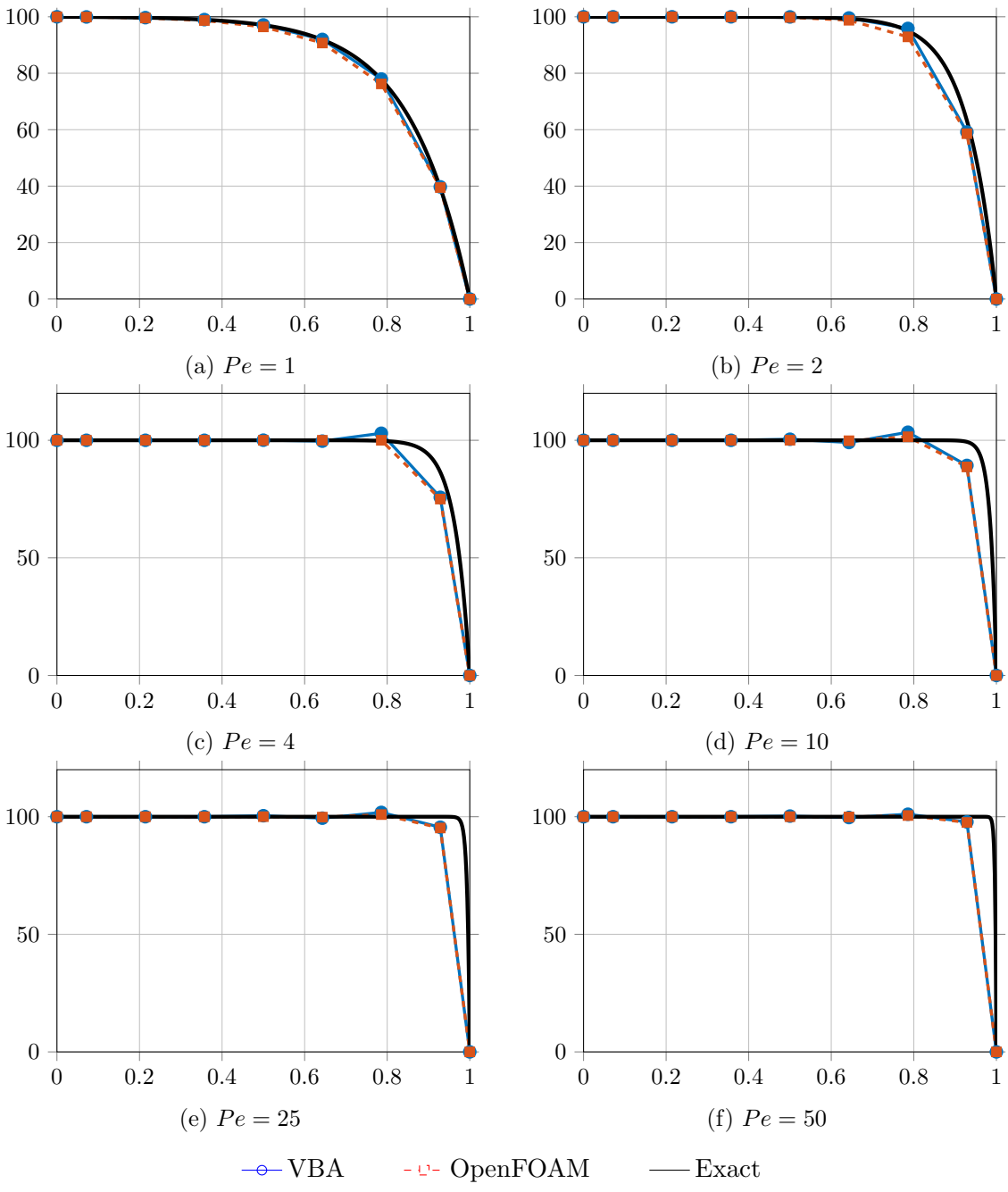


Figure 5.4: Comparison of VBA, OpenFOAM, and exact solutions for the QUICK scheme.

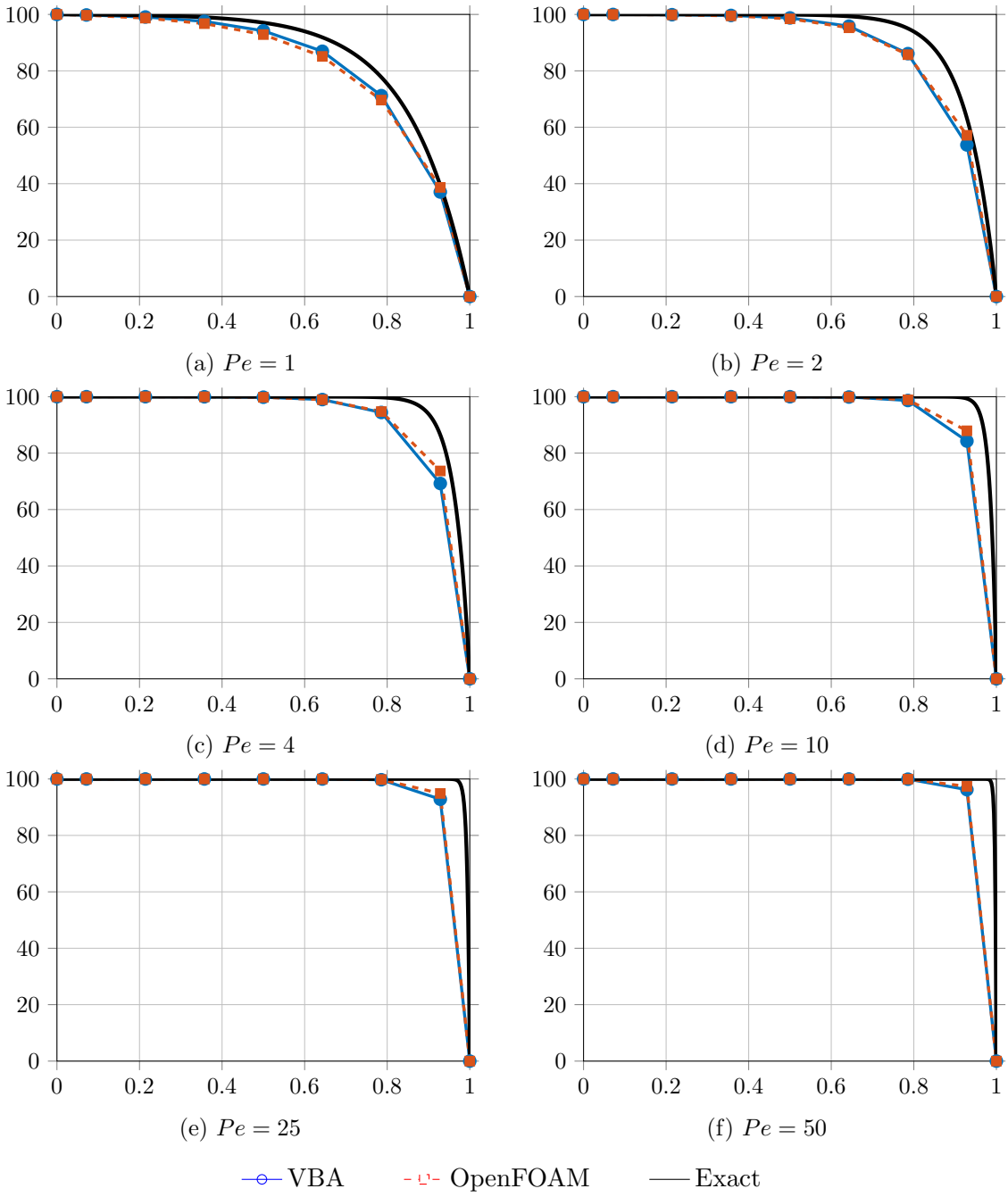


Figure 5.5: Comparison of VBA, OpenFOAM, and exact solutions for the MUSCL scheme.

Chapter 6

Conclusion

This report presented the derivation, implementation, and validation of a 1D finite volume solver for the steady convection-diffusion equation. Eight discretization schemes were derived from first principles in Chapter 3, with full treatment of internal and boundary cells for each. Chapter 4 described how these derivations translate directly into a structured VBA implementation, where a single coefficient representation shared by all seven linear schemes allows one generic Gauss-Seidel solver to handle them all, with MUSCL handled separately through a deferred-correction approach.

The validation in Chapter 5 showed that the VBA solver and OpenFOAM agree to within reporting precision for CDS, UDS, and SOU across the full range of Péclet numbers tested. QUICK and MUSCL show close but not exact agreement with their OpenFOAM counterparts.

This validation confirms that the discretization derived in Chapter 3 and implemented in Chapter 4 is not just self-consistent, but matches what an established, peer-reviewed CFD code produces for the same problem under the same assumptions.

The solver in its current form covers the full set of classical 1D schemes for the steady convection-diffusion problem, and is organized to be readable, debuggable, and extensible. Natural directions for further work include extending the solver to unsteady problems, two-dimensional domains, non-uniform grids, and coupled systems of transported scalars.

Bibliography

[1] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere Publishing Corporation, 1980.